

Chapter 15: Advanced Classes

By Victoria Gonda

An earlier chapter introduced you to the basics of defining and using classes in Kotlin. Classes are used to support traditional object-oriented programming.

Class concepts include inheritance, overriding, polymorphism and composition which makes them suited for this purpose. These extra features require special consideration for construction, class hierarchies, and understanding the class lifecycle in memory.

This chapter will introduce you to the finer points of classes in Kotlin, and help you understand how you can create more complex classes. Open up the starter project or a new Kotlin project to get started.

Introducing inheritance

In an earlier chapter, you saw a `Grade` class and a pair of class examples: `Person` and `Student`.

```
data class Grade(  
    val letter: Char,  
    val points: Double,  
    val credits: Double  
)  
  
class Person(var firstName: String, var lastName: String) {  
    fun fullName() = "$firstName $lastName"  
}  
  
class Student(var firstName: String, var lastName: String,  
    var grades: MutableList<Grade> = mutableListOf<Grade>()) {  
    fun recordGrade(grade: Grade) {  
        grades.add(grade)  
    }  
}
```

There's an incredible amount of redundancy between `Person` and `Student`. They share many of the same properties. Maybe you've also noticed that a `Student` is a `Person`!

This simple case demonstrates the idea behind **class inheritance**. Much like in the real world, where you can think of a student as a person, you can represent the same relationship in code by replacing the original `Person` and `Student` class implementations with the following. Add these classes to your code:

```
// 1  
open class Person(var firstName: String, var lastName: String) {  
    fun fullName() = "$firstName $lastName"  
}  
  
// 2  
class Student(  
    firstName: String,  
    lastName: String,  
    var grades: MutableList<Grade> = mutableListOf<Grade>()  
) : Person(firstName, lastName) {  
    open fun recordGrade(grade: Grade) {  
        grades.add(grade)  
    }  
}
```

In this modified example:

1. The Person class now includes the open keyword.
2. The Student class now **inherits** from Person, indicated by a colon after the naming of Student, followed by the class from which Student inherits, which in this case is Person.

The open keyword means that the Person class is *open* to be inherited from; the need for open is part of the Kotlin philosophy of requiring choices such as inheritance to be explicitly defined by the programmer.

You must still add parameters such as `firstName` to the Student constructor, and they are then passed along as arguments to the Person constructor. Notice in the modified example that the `var` keyword is no longer needed on the parameters, since they are already defined as properties in the Person class.

Through inheritance, Student automatically gets the properties and methods declared in the Person class. In code, it would be accurate to say that a Student *is-a* Person.

With much less duplication of code, you can now create Student objects that have all the properties and methods of a Person. Add this to `main()`:

```
val john = Person(firstName = "Johnny", lastName = "Appleseed")
val jane = Student(firstName = "Jane", lastName = "Appleseed")

john.fullName() // Johnny Appleseed
jane.fullName() // Jane Appleseed
```

Both `john` and `jane` have all the properties of a Person because Student inherits from Person.

Additionally, only the Student object will have all of the properties and methods defined in Student. Try this out:

```
val history = Grade(letter = 'B', points = 9.0, credits = 3.0)
jane.recordGrade(history)
// john.recordGrade(history) // john is not a student!
```

You can't record a grade for `john` because he's not a student. The shared properties only go one direction.

A class that inherits from another class is known as a **subclass** or a **derived class**, and the class from which it inherits is known as a **superclass** or **base class**.

The rules for subclassing are fairly simple:

- A Kotlin class can inherit from only one other class, a concept known as **single inheritance**.
- A Kotlin class can only inherit from a class that is **open**.
- There's no limit to the depth of subclassing, meaning you can subclass from a class that is *also* a subclass, like below (and first redefining Student with open):

```
open class Student(  
    firstName: String,  
    lastName: String,  
    var grades: MutableList<Grade> = mutableListOf<Grade>()  
) : Person(firstName, lastName) {  
  
    open fun recordGrade(grade: Grade) {  
        grades.add(grade)  
    }  
}  
  
open class BandMember(  
    firstName: String,  
    lastName: String  
) : Student(firstName, lastName) {  
    open val minimumPracticeTime: Int  
        get() { return 2 }  
}  
  
class OboePlayer(  
    firstName: String,  
    lastName: String  
) : BandMember(firstName, lastName) {  
    // This is an example of an override, will be covered soon.  
    override val minimumPracticeTime: Int =  
        super.minimumPracticeTime * 2  
}
```

A chain of subclasses is called a **class hierarchy**. In this example, the hierarchy would be OboePlayer → BandMember → Student → Person. A class hierarchy is analogous to a family tree. Because of this analogy, a superclass is also called the **parent class** of its **child class**.

Polymorphism

The Student–Person relationship demonstrates a computer science concept known as **polymorphism**. In brief, polymorphism is a programming language’s ability to treat an object differently based on context.

An OboePlayer is of course an OboePlayer, but it is also a Person. Because it derives from Person, you could use an OboePlayer object anywhere you’d use a Person object.

This example demonstrates how you can treat an OboePlayer as a Person. Add this to your example:

```
fun phonebookName(person: Person): String {  
    return "${person.lastName}, ${person.firstName}"  
}  
  
val person = Person(  
    firstName = "Johnny",  
    lastName = "Appleseed"  
)  
val oboePlayer = OboePlayer(  
    firstName = "Jane",  
    lastName = "Appleseed"  
)  
  
phonebookName(person)    // Appleseed, Johnny  
phonebookName(oboePlayer) // Appleseed, Jane
```

Because OboePlayer derives from Person, it is a valid input into the function `phonebookName()`. More importantly, the function has no idea that the object passed in is anything *other* than a regular Person. It can only observe the elements of OboePlayer that are defined in the Person base class.

With the polymorphism characteristics provided by class inheritance, Kotlin is treating the object pointed to by `oboePlayer` differently based on the context. This can be particularly useful to you when you have diverging class hierarchies, but want to have code that operates on a common type or base class.

For example, if you also have a hierarchy `SoccerPlayer → StudentAthlete → Student → Person`, and you want code that operates on all Students, you can do this regardless of if a student is an OboePlayer or a SoccerPlayer.

Runtime hierarchy checks

Now that you are coding with polymorphism, you will likely find situations where the specific type behind a variable can be different. For instance, you could define a variable `hallMonitor` as a `Student`. Add a `hallMonitor` to `main()`:

```
var hallMonitor =  
    Student(firstName = "Jill", lastName = "Bananapeel")
```

But what if `hallMonitor` were a more derived type, such as an `OboePlayer`? Change `hallMonitor` to be an `oboePlayer`.

```
hallMonitor = oboePlayer
```

Because `hallMonitor` is originally defined as a `Student`, the compiler won't allow you to attempt calling properties or methods for a more derived type. You'll get an error if you try to do something like this:

```
hallMonitor.minimumPracticeTime // Error!
```

You get an error here because the compiler only knows that `hallMonitor` is a `Student`. It does not know `hallMonitor` is a `BandMember` or `OboePlayer`.

Fortunately, Kotlin gives you the `is` operator to check whether an instance is part of a given inheritance hierarchy, and `!is` (not-is) for the opposite. Add these checks:

```
println(hallMonitor is OboePlayer) // true, since assigned it to  
oboePlayer  
println(hallMonitor !is OboePlayer) // also have !is for "not-  
is"  
println(hallMonitor is Person) // true, because Person is  
ancestor of OboePlayer
```

Run your code to see the result.

Kotlin also provides the `as` infix operator to treat a property or a variable as another type:

- `as`: An **unsafe** cast to a specific type that is known at compile time to succeed, such as casting to a supertype.
- `as?`: A **safe** cast (to a subtype). If the cast fails, the result of the expression will be `null`.

These can be used in various contexts to treat the `hallMonitor` as a `BandMember`, or the `oboePlayer` as a less-derived `Student`:

```
(oboePlayer as Student).minimumPracticeTime // Error: No longer
a band member!

(hallMonitor as? BandMember)?.minimumPracticeTime
// 4 if hallMonitor = oboePlayer, else null
```

You may be wondering under what contexts you would use the `as` operator by itself. Any object contains all the properties and methods of its parent class, so what use is casting it to something it already is?

Kotlin has a strong type system, and the interpretation of a specific type can have an effect on **static dispatch**, or the decision of which specific operation is selected at compile time. Sound confusing? How about an example?

Assume you have two functions with identical names and parameter names for two different parameter types:

```
fun afterClassActivity(student: Student): String {
    return "Goes home!"
}

fun afterClassActivity(student: BandMember): String {
    return "Goes to practice!"
}
```

If you were to pass `oboePlayer` into `afterClassActivity()`, which one of these implementations would get called? The answer lies in Kotlin's dispatch rules, which in this case will select the more specific version that takes in an `OboePlayer`.

If you were to cast `oboePlayer` to a `Student`, the `Student` version would be called:

```
afterClassActivity(oboePlayer) // Goes to practice!
afterClassActivity(oboePlayer as Student) // Goes home!
```

Inheritance, methods and overrides

Subclasses' properties and methods are defined in their superclass, plus any additional properties and methods the subclass defines for itself. In that sense, subclasses are *additive*; for example, you've already seen that the `Student` class can add additional properties and methods for handling a student's grades. These properties and methods wouldn't be available to any `Person` class instances, but they *would* be available to `Student` subclasses.

Besides creating their own methods, subclasses can **override** methods defined in their superclass. Assume that student athletes become ineligible for the athletics program if they're failing three or more classes. That means you need to keep track of failing grades somehow. Add this class to your file:

```
class StudentAthlete(
    firstName: String,
    lastName: String
): Student(firstName, lastName) {
    val failedClasses = mutableListOf<Grade>()

    override fun recordGrade(grade: Grade) {
        super.recordGrade(grade)

        if (grade.letter == 'F') {
            failedClasses.add(grade)
        }
    }

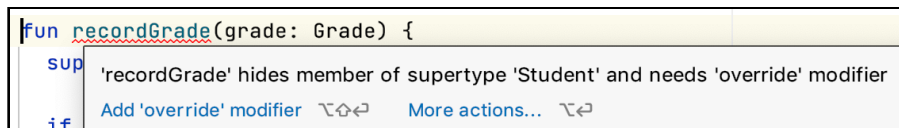
    val isEligible: Boolean
    get() = failedClasses.size < 3
}
```

Take a close look at `recordGrade()`. In this example, the `StudentAthlete` class overrides `recordGrade()` so it can keep track of any courses the student has failed.

The `StudentAthlete` class then has its own computed property, `isEligible`, that uses this information to determine the athlete's eligibility.

When overriding a method, use the `override` keyword before the method declaration.

If your subclass were to have an identical method declaration as its superclass, but you omitted the `override` keyword, Kotlin would indicate a build error.



recordGrade hides member of supertype Student and needs override modifier

This makes it very clear whether a method is an override of an existing one or not.

Create an instance of the subclass and make calls to both the overridden and new methods:

```
val math = Grade(letter = 'B', points = 9.0, credits = 3.0)
val science = Grade(letter = 'F', points = 9.0, credits = 3.0)
```



```
val physics = Grade(letter = 'F', points = 9.0, credits = 3.0)
val chemistry = Grade(letter = 'F', points = 9.0, credits = 3.0)

val dom = StudentAthlete(firstName = "Dom", lastName = "Grady")
dom.recordGrade(math)
dom.recordGrade(science)
dom.recordGrade(physics)
println(dom.isEligible) // > true
dom.recordGrade(chemistry)
println(dom.isEligible) // > false
```

Run this code so you can see your changes in action.

Introducing super

You may have also noticed the line `super.recordGrade(grade)` in the overridden method. The `super` keyword is similar to `this`, except it will invoke the method in the nearest implementing superclass. In the example of `recordGrade()` in `StudentAthlete`, calling `super.recordGrade(grade)` will execute the method as defined in the `Student` class.

Remember how inheritance let you define `Person` with first name and last name properties and avoid repeating those properties (using `val` or `var`) in subclasses? Similarly, being able to call the superclass methods means you can write the code to record the grade once in `Student` and then call “up” to it as needed in subclasses.

Although it isn’t always required, it’s often important to call `super` when overriding a method in Kotlin. The `super` call is what will record the grade itself in the `grades` array, because that behavior isn’t duplicated in `StudentAthlete`. Calling `super` is also a way of avoiding the need for duplicate code in `StudentAthlete` and `Student`.

When to call super

As you may notice, exactly *when* you call `super` can have an important effect on your overridden method.

Suppose you replace the overridden `recordGrade()` method in the `StudentAthlete` class with the following version that recalculates the `failedClasses` each time a grade is recorded:

```
override fun recordGrade(grade: Grade) {
    var newFailedClasses = mutableListOf<Grade>()
    for (grade in grades) {
        if (grade.letter == 'F') {
            newFailedClasses.add(grade)
        }
    }
}
```

```
    }  
  }  
  failedClasses = newFailedClasses  
  super.recordGrade(grade)  
}
```

This version of `recordGrade()` uses the `grades` array to find the current list of failed classes. If you've spotted a bug in the code above, good job! Since you call `super` last, if the new `grade.letter` is an `F`, the code won't update `failedClasses` properly.

While it's not a hard rule, it's generally best practice to call the `super` version of a method first when overriding. That way, the superclass won't experience any side effects introduced by its subclass, and the subclass won't need to know the superclass's implementation details.

Preventing inheritance

Often you'll want to disallow subclasses of a particular class. Kotlin makes this easy since the default for `class` definitions is that classes are **not** open to subclassing; you **must** use the `open` keyword to allow inheritance.

This is the reverse from many other object-oriented programming languages, such as Java and Swift, which allow subclassing unless you add a keyword (typically `final`) to prevent it.

Here's an example in Kotlin:

```
class FinalStudent(firstName: String, lastName: String)  
    : Person(firstName, lastName)  
  
class FinalStudentAthlete(firstName: String, lastName: String)  
    : FinalStudent(firstName, lastName) // Build error!
```

This causes an error when trying to define `FinalStudentAthlete` because `FinalStudent` is not marked as `open`.

By not marking the `FinalStudent` class `open`, you tell the compiler to prevent any classes from inheriting from `FinalStudent`. Kotlin is designed to improve your use of inheritance by only allowing you to inherit when you specifically want to.

The Kotlin approach is similar with respect to overriding functions in classes. If you only want specific methods to be overridden, you can mark those methods as open:

```
open class AnotherStudent(firstName: String, lastName: String)
    : Person(firstName, lastName) {

    open fun recordGrade(grade: Grade) {}
    fun recordTardy() {}
}

class AnotherStudentAthlete(firstName: String, lastName: String)
    : AnotherStudent(firstName, lastName) {

    override fun recordGrade(grade: Grade) {} // OK
    override fun recordTardy() {} // Build error! recordTardy is
final
}
```

Here, you get an error when trying to override `recordTardy()` because it's not open in the superclass.

Kotlin's approach of defaulting to classes and methods being final tells the compiler it doesn't need to look for any more subclasses, which can shorten compile time, and it also requires you to be very explicit when deciding to allow a class to be inherited from.

Abstract classes

In certain situations, you may want to prevent a class from being instantiated, but still be able to be inherited from. This will let you define properties and behavior common to all subclasses. You can only create instances of the subclasses and not the base, parent class. Such parent classes are called **abstract**.

Classes declared with the abstract keyword are **open** by default and can be inherited from. In abstract classes, you can also declare abstract methods marked with abstract that have no body. The abstract methods must be overridden in subclasses.

Add these classes:

```
abstract class Mammal(val birthDate: String) {  
    abstract fun consumeFood()  
}  
  
class Human(birthDate: String): Mammal(birthDate) {  
    override fun consumeFood() {  
        // ...  
    }  
    fun createBirthCertificate() {  
        // ...  
    }  
}
```

Here, you define a `Mammal` class, which is abstract, and a `Human` class, which inherits from `Mammal`.

Now, see what happens when you try to create an instance of each of these:

```
val human = Human("1/1/2000")  
val mammal = Mammal("1/1/2000") // Error: Cannot create an  
instance of an abstract class
```

You can create an instance of the `Mammal` subclass `Human`, but not of the `Mammal` class itself.

Abstract classes are closely related to **interfaces**, which you'll learn about in Chapter 17: "Interfaces."

Sealed classes

Sealed classes are useful when you want to make sure that the values of a given type can only come from a particular limited set of subtypes. They allow you to define a strict hierarchy of types. The sealed classes themselves are abstract and cannot be instantiated.

Sealed classes act very much like **enum classes**, which you'll learn about in the next chapter, but also allow subtypes which can have multiple instances and have state.

Add a sealed class `Shape` that has subtypes `Circle` and `Square`:

```
sealed class Shape {  
    class Circle(val radius: Int): Shape()  
    class Square(val sideLength: Int): Shape()  
}
```

You've used the keyword `sealed` to mark `Shape` as a sealed class. Both circles and squares are shapes, but a circle has a radius and a square has a side length.

Unlike enum classes, you can create multiple instances of each type within the sealed class. Create these instances:

```
val circle1 = Shape.Circle(4)
val circle2 = Shape.Circle(2)
val square1 = Shape.Square(4)
val square2 = Shape.Square(2)
```

You're able to make as many Circles and Squares as you like, but no Shapes.

Functions defined on `Shape` can distinguish between the different subtypes using a `when` expression. Add this function:

```
fun size(shape: Shape): Int {
    return when (shape) {
        is Shape.Circle -> shape.radius
        is Shape.Square -> shape.sideLength
    }
}

size(circle1) // radius of 4
size(square2) // sideLength of 2
```

You'll learn more about sealed classes in Chapter 16, "Enum & Sealed Classes."

Secondary constructors

You've seen how to define the primary constructors of classes, by appending a list of property parameters and their types to the class name.

The keyword `constructor` was implicit in the primary constructor:

```
class Person(var firstName: String, var lastName: String) {
    fun fullName() = "$firstName $lastName"
}

// is the same as

class Person constructor(var firstName: String, var lastName:
String) {
    fun fullName() = "$firstName $lastName"
}
```

Both of the above definitions work the same, even though one uses the constructor keyword.

You can also use the constructor keyword to define secondary constructors for a class, within the class body. You can call between the various constructors using the `this` keyword:

```
open class Shape {  
    constructor(size: Int) {  
        // ...  
    }  
  
    constructor(size: Int, color: String) : this(size) {  
        // ...  
    }  
}
```

In this case, one secondary constructor is calling another with a single `Int` argument representing the size.

When subclassing, you can call from constructors in the subclass to constructors in the superclass using `super`:

```
class Circle : Shape {  
    constructor(size: Int) : super(size) {  
        // ...  
    }  
  
    constructor(size: Int, color: String) : super(size, color) {  
        // ...  
    }  
}
```

Nested and inner classes

When two classes are closely related to each other, sometimes it's useful to define one class within the scope of another class. By doing so, you've namespaced one class within the other.

Add `Car` and `Engine` to your project:

```
class Car(val carName: String) {  
    class Engine(val engineName: String)  
}
```

Other classes that want to use the Engine class must refer to it as `Car.Engine`. In this case, Engine is a **nested class** of Car.

When a class is nested inside another, it does not by default have access to the other members of the class:

```
class Car(val carName: String) {  
    class Engine(val engineName: String) {  
        override fun toString(): String {  
            return "$engineName in a $carName" // Error: cannot see  
            carName in outer scope!  
        }  
    }  
}
```

Since `carName` is a property of `Car`, it is not accessible from the nested class `Engine`.

If you want the nested class to have access to the other members, you need to define it with the `inner` keyword.

Update your Engine to look like this:

```
inner class Engine(val engineName: String) {  
    override fun toString(): String {  
        return "$engineName engine in a $carName"  
    }  
}
```

Since `Engine` is now an inner class of `Car`, it can access the other members of `Car`

Add and run this code:

```
val mazda = Car("mazda")  
val mazdaEngine = mazda.Engine("rotary")  
println(mazdaEngine) // > rotary engine in a mazda
```

The engine can now get the car type just fine!

Visibility modifiers

While the `open` keyword determines what you can and cannot override in class hierarchies, **visibility modifiers** determine what can and cannot be seen both inside and outside of classes.

The four visibility modifiers available in Kotlin are:

- **public**: Visible from everywhere, within subclasses, other files, and other project modules; if no visibility modifier is specified, it defaults to **public**.
- **private**: Visible only within the same class for classes, and only within the same file for top-level functions and other non-class definitions. This means hidden from subclasses, too.
- **protected**: Visible only within subclasses for class hierarchies.
- **internal**: Visible only within the same **module**, for example, an IntelliJ IDEA module.

Generally you want to limit the visibility or **scope** of your classes and variables as much as possible. This will keep the responsibility of your classes clear and prevent you from changing the state of a class when you really shouldn't be.

Add a class hierarchy consisting of a `User` and a `PrivilegedUser` with a list of privileges:

```
data class Privilege(val id: Int, val name: String)

open class User(
    val username: String,
    private val id: String,
    protected var age: Int
)

class PrivilegedUser(username: String, id: String, age: Int)
    : User(username, id, age) {
    private val privileges = mutableListOf<Privilege>()

    fun addPrivilege(privilege: Privilege) {
        privileges.add(privilege)
    }

    fun hasPrivilege(id: Int): Boolean {
        return privileges.map { it.id }.contains(id)
    }

    fun about(): String {
        //return "$username, $id" // Error: id is private
        return "$username, $age" // OK: age is protected
    }
}
```


In the super class, `User`, the `id` property is marked `private`, so can only be referenced inside the `User` class. The `age` property is `protected`, so the subclass `PrivilegedUser` can see it.

Try it out:

```
val privilegedUser =
    PrivilegedUser(username = "sashinka", id = "1234", age = 21)
val privilege = Privilege(1, "invisibility")
privilegedUser.addPrivilege(privilege)
println(privilegedUser.about()) // > sashinka, 21
```

`PrivilegedUser` can access both the `username` property, which is `public`, and the `age` property.

When and why to subclass

This chapter has introduced you to class inheritance, along with the numerous programming techniques that subclassing enables. But you might be asking, “When should I subclass?”

Rarely is there a right or wrong answer to that important question. Understanding the trade-offs can help you make the best decision for any particular case.

Using the `Student` and `StudentAthlete` classes as an example, you might decide you can simply put all of the characteristics of `StudentAthlete` into `Student`:

```
data class Sport(val name: String)

class Student2(firstName: String, lastName: String)
    : Person(firstName, lastName) {
    var grades = mutableListOf<Grade>()
    var sports = mutableListOf<Sport>()
    // original code
}
```

In reality, this *could* solve all of the use cases for your needs. A `Student2` that doesn’t play sports would simply have an empty `sports` array, and you would avoid some of the added complexities of subclassing.

Single responsibility

In software development, however, the guideline known as the **single responsibility principle** states that any class should have a single concern. In `StudentAthlete`, you might argue that it shouldn't be the `Student` class's job to encapsulate responsibilities that only make sense to student athletes, and it makes sense to create the `StudentAthlete` subclass rather than keep a list of sports within `Student`.

Strong types

Subclassing creates an additional type. With Kotlin's type system, you can declare properties or behavior based on objects that are student athletes, not regular students:

```
class Team {  
    var players = mutableListOf<StudentAthlete>()  
  
    val isEligible: Boolean  
    get() {  
        for (player in players) {  
            if (!player.isEligible) {  
                return false  
            }  
        }  
        return true  
    }  
}
```

A team has players who are student athletes. If you tried to add a regular `Student` object to the array of players, the type system wouldn't allow it. This can be useful as the compiler can help you enforce the logic and requirement of your system.

Shared base classes

You can subclass a shared base class multiple times by classes that have mutually exclusive behavior:

```
// A button that can be pressed.  
open class Button {  
    fun press() {  
    }  
}  
  
// An image that can be rendered on a button.
```

```
class Image

// A button that is composed entirely of an image.
class ImageButton(var image: Image): Button()

// A button that renders as text.
class TextButton(val text: String): Button()
```

In this example, you can imagine numerous `Button` subclasses that share only the fact that they can be pressed. The `ImageButton` and `TextButton` classes likely have entirely different mechanisms to render the appearance of a button, so they might have to implement their own behavior when the button is pressed.

You can see here how storing image and text in the `Button` class — not to mention any other kind of button there might be — would quickly become impractical. It makes sense for `Button` to be concerned with the press behavior, and the subclasses to handle the actual look and feel of the button.

Extensibility

Sometimes you simply must subclass if you're extending the behavior of code you don't own. In the example above, it's possible `Button` is part of a framework you're using, and there's no way you can modify or extend the source code to fit your needs.

In that case, subclass `Button` so you can add your custom subclass and use it with code that's expecting an object of type `Button`. As you've seen earlier in this chapter, the author of a class can designate if any of the members of a class can be overridden or not using the `open` keyword.

Note: When should you subclass versus use an extension function? This will vary depending on use case and team practices. Explore both options and see which is best for your needs.

Identity

Finally, it's important to understand that classes and class hierarchies model what objects *are*. If your goal is to share behavior (what objects *can do*) between types, more often than not you should prefer **interfaces** over subclassing. You'll learn about interfaces in Chapter 17: "Interfaces".

Challenges

1. Create three simple classes called A, B, and C where C inherits from B and B inherits from A. In each class initializer, call `println("I'm <X>!")` where X is the name of the class. Create an instance of C called c. What order do you see each `println()` called in?
2. Cast the instance of type C to an instance of type A. Which casting operation do you use and why? Create an instance of A called a. What happens if you try to cast a to C?
3. Create a subclass of `StudentAthlete` called `StudentBaseballPlayer` and include properties for `position`, `number`, and `battingAverage`. What are the benefits and drawbacks of subclassing `StudentAthlete` in this scenario?
4. Create a sealed class `Resource` with subtypes `Success`, `Loading`, and `Error`. Give the `Success` type a string data property and the `Error` type a string error property. Can you imagine a use for this `Resource` type?

Key points

- **Class inheritance** is one of the most important features of classes and enables **polymorphism**.
- **Subclassing** is a powerful tool, but it's good to know when to subclass. Subclass when you want to extend an object and could benefit from an “is-a” relationship between subclass and superclass, but be mindful of the inherited state and deep class hierarchies.
- The **open** keyword is used to allow inheritance from classes and also to allow methods to be overridden in subclasses.
- **Sealed classes** allow you to create a strictly defined class hierarchy that is similar to an enum class but that allow multiple instances of each subtype to be created and hold state.
- **Secondary constructors** allow you to define additional constructors that take additional parameters than the primary constructor and take different actions with those parameters.
- **Nested classes** allow you to namespace one class within another.
- **Inner classes** are nested classes that also have access to the other members of the outer class.
- **Visibility modifiers** allow you to control where class members and top-level declarations can be seen within your code and projects.

Where to go from here?

Classes are the programming construct you will most often use to model things in your Kotlin apps, from students to grades to people and much more. Classes allow for the definition of hierarchies of items and also for one type of item to be **composed** within another.

In the next chapter, you'll learn about another special type of class called an **enum class**.